SESSION 04 / PRINTABLE GUIDE

Lesson 5: Explore first, then visualize what changed

Session 04 | Notebook exploration and visualization
Output: A small data profile, three diagnostic visualizations, and evidence-backed cleaning decisions.

Micro-fixture explanation: These charts deliberately fit nine raw events. For the main participant exercise, use full-batch exploration and fourteen-day scenario visualizations. Those include daily patterns, a firmware association, source gaps, and explicit accepted-coverage denominators rather than implying trends from two points.

Where this fits

Use the device example as the common story. Run its Cell 1 to create Bronze, pause for Part A here, and try the Data Wrangler walkthrough. Then finish the device example's Cells 2-4 and return for Part B.

These are notebook cells, not another ingestion job. They read the isolated teaching tables and do not overwrite them. Use the same approved Lakehouse context.

The teaching question is not "Which chart looks best?" It is "What does this evidence tell us about the source, acceptance rules, and intended output?"

Part A: Explore Bronze before changing it

1. Inspect shape, schema, and representative rows

PYTHON

```python
from pyspark.sql import functions as F

spark.conf.set("spark.sql.session.timeZone", "UTC")
raw = spark.table("s04_device_events_bronze").filter(
    F.col("batch_id") == "s04-devices-001"
)

raw.printSchema()
print("Raw event rows:", raw.count())
raw.orderBy(
    F.expr("try_cast(export_row_number as int)")
).limit(9).show(truncate=False)
```

Expect nine rows. The nine source columns are strings at landing; the table also has batch, source-file, and ingestion metadata. A preview is useful for noticing shapes and values, but it does not prove the absence of rare problems.

Ask participants to identify:

- The likely business key: device and event, not export row position.
- Values that look numeric but are still text.
- Technical context worth preserving: firmware and signal strength.
- Presentation/debug fields with no role in this analytical contract.

2. Separate missingness from validity

PYTHON

```python
source_columns = [
    "device_id", "event_id", "event_time_utc", "temperature_c",
    "firmware_version", "signal_strength_dbm",
    "ui_theme", "debug_message", "export_row_number",
]

missing_counts = raw.agg(*[
    F.sum(
        F.when(
            F.col(name).isNull() | (F.length(F.trim(F.col(name))) == 0),
            1,
        ).otherwise(0)
    ).alias(name + "_missing")
    for name in source_columns
])
missing_counts.show(truncate=False)

profiled = (
    raw.withColumn(
        "temperature_value", F.expr("try_cast(temperature_c as decimal(8,2))")
    )
    .withColumn(
        "event_timestamp", F.expr("try_cast(event_time_utc as timestamp)")
    )
)

profiled.agg(
    F.count("*").alias("raw_rows"),
    F.sum(F.when(F.col("temperature_value").isNull(), 1).otherwise(0))
    .alias("unusable_temperature"),
    F.sum(F.when(F.col("event_timestamp").isNull(), 1).otherwise(0))
    .alias("unusable_timestamp"),
    F.sum(
        F.when(
```

```
            F.col("temperature_value").isNotNull()
            & ~F.col("temperature_value").between(-40, 85),
            1,
        ).otherwise(0)
    ).alias("temperature_outside_range"),
).show()
```

All source missing counts are zero, yet there is one unusable temperature, one unusable timestamp, and one out-of-range reading. `sensor error`, `not a timestamp`, and `200.00` are present values, not missing cells.

This profile illustrates those specific conditions; it is not the complete acceptance function. The device lesson additionally checks the UTC text format, keys, duplicate policy, and numeric format. Do not copy a partial profiling expression into production as the full rule.

3. Check identity and duplicate meaning

PYTHON

```python
raw.agg(
    F.countDistinct("device_id").alias("raw_device_spellings"),
    F.countDistinct(F.upper(F.trim("device_id"))).alias("normalized_devices"),
).show()

duplicate_keys = (
    raw.select(
        F.upper(F.trim("device_id")).alias("device_id"),
        F.upper(F.trim("event_id")).alias("event_id"),
    )
    .groupBy("device_id", "event_id").count()
    .filter(F.col("count") > 1)
)
duplicate_keys.show()
```

Expect three raw device spellings but two normalized devices. The repeated business key is D01/E002 with two occurrences. One duplicate-key group, two ambiguous source rows, and one excess occurrence are different counts; label the one you mean.

`export_row_number` is unique from 1 to 9, but it does not make the event key unique. Including it in the duplicate key would hide the issue.

4. Inspect the distribution without declaring every extreme wrong

```python
profiled.select("temperature_value").summary(
    "count", "min", "max", "mean"
).show()

display(
    profiled.select(
        "device_id", "event_id", "temperature_value",
        "event_timestamp", "firmware_version", "signal_strength_dbm",
    ).limit(1000)
)
```

Eight raw values can be converted to numbers. Their minimum is 20.00, maximum 200.00, and mean 46.875. That mean includes duplicated events and a numeric reading with an invalid timestamp. It is a raw diagnostic, not a valid published business measure.

In Fabric's result widget, use the table/Inspect view to examine column statistics. A numeric histogram can expose the extreme 200.00 value. The agreed device range, not the shape of the histogram alone, justifies rejection. Do not clip it to 85 or replace it with an average merely because it looks inconvenient.

The display widget can show only a bounded view unless full-result aggregation is explicitly configured. These nine rows fit; a large source requires full-data Spark profiles plus a clearly labeled sample for inspection.

Part B: Visualize the accepted and rejected results

First finish the device example's cleaning and daily-table cells. The accepted Silver table contains four events, quarantine five rows, and the transfer table two device/day summaries.

Chart 1: Which rules rejected rows?

```python
rejection_counts = (
    spark.table("s04_device_events_rejected")
    .filter(F.col("batch_id") == "s04-devices-001")
    .groupBy("quality_issue").count()
    .orderBy("quality_issue")
)
display(rejection_counts)
```

Choose a bar chart in the output widget: X = `quality_issue`, Y = `count`. If the chart requires an aggregation, sum the already grouped count. Do not count the four summary rows and mistakenly show four bars of height one.

Expected heights: `DUPLICATE_EVENT` = 2; `INVALID_TEMPERATURE` = 1; `INVALID_TIMESTAMP` = 1; `OUT_OF_RANGE` = 1.

These are primary rejection classifications. If rules overlap in a different batch, the chart follows the chosen precedence; it does not count every possible condition independently.

Chart 2: Accepted temperature over UTC time

The following optional code-based visualization uses pandas and Matplotlib from the approved runtime. Verify they are available before delivery; do not install unreviewed packages during a customer session.

PYTHON

```python
import pandas as pd
import matplotlib.pyplot as plt

accepted_plot = (
    spark.table("s04_device_events_silver")
    .filter(F.col("batch_id") == "s04-devices-001")
    .select("device_id", "event_time_utc", "temperature_c")
    .orderBy("device_id", "event_time_utc")
    .limit(1000)
    .toPandas()
)
accepted_plot["event_time_utc"] = pd.to_datetime(
    accepted_plot["event_time_utc"], utc=True
)

fig, ax = plt.subplots(figsize=(9, 4))
for device, points in accepted_plot.groupby("device_id"):
    ax.plot(
        points["event_time_utc"],
        points["temperature_c"].astype(float),
        marker="o",
        label=device,
    )
ax.set(
    title="Accepted observations only - s04-devices-001",
    xlabel="Event time (UTC)",
    ylabel="Temperature (degrees C)",
)
ax.legend(title="Device")
fig.autofmt_xdate()
fig.tight_layout()
plt.show()
plt.close(fig)
```

Expect D01 observations at 20/24 degrees and D02 at 30/32. The connecting line is a visual aid, not proof of continuous measurement or a value between observations. There are only two accepted readings per device; do not claim a meaningful long-term trend.

`limit(1000)` occurs before `toPandas()` so the driver does not collect an unbounded table. It is a cap, not a representative sampling method. For large inputs, filter to an approved time/device scope or aggregate in Spark first, then label the chart's scope.

Chart 3: Daily metrics and their denominator

PYTHON

```python
daily_plot = (
    spark.table("s04_device_daily_transfer")
    .filter(F.col("batch_id") == "s04-devices-001")
    .select("device_id", "reading_date", "reading_count", "avg_temperature_c")
    .orderBy("reading_date", "device_id")
    .limit(1000)
    .toPandas()
)

fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(
    daily_plot["device_id"],
    daily_plot["avg_temperature_c"].astype(float),
)
for bar, readings in zip(bars, daily_plot["reading_count"]):
    ax.annotate(
        f"{int(readings)} accepted",
        (bar.get_x() + bar.get_width() / 2, bar.get_height()),
        ha="center",
        va="bottom",
    )
ax.set(
    title="Accepted-reading averages - 2026-10-28 UTC",
    xlabel="Device",
    ylabel="Average temperature (degrees C)",
)
fig.tight_layout()
plt.show()
plt.close(fig)
```

Expect 22.00 for D01 and 31.00 for D02, each based on two accepted readings. This plots the notebook's transfer result; it is not evidence that Warehouse Gold has loaded. After Warehouse publication, query Gold and compare the same values.

This fixture has one day. For multiple days, filter to a stated day or make the date an explicit grouping/facet; do not overlay several device/day bars and silently imply one result per device.

Teaching exercise and completion evidence

Ask learners to submit:

1. A profile distinguishing missing, unparseable, out-of-range, and duplicate conditions.

2. One annotated diagnostic chart and the precise question it answers.
3. One cleaning decision supported by source/business evidence rather than appearance.
4. An explanation of raw rows (9), accepted events (4), quarantine rows (5), and daily results (2).
5. A sample/full-data label and the scope/date used for visualization.

Do not treat an attractive chart as an acceptance test or a notebook chart as a replacement for governed customer reporting.

Next: Data Wrangler and Silver-to-Gold column design.

Reference: Fabric notebook visualization, reviewed October 29, 2026. Product documentation is not proof of current GCC High UI availability.

Source: 05_exploration_and_visualization.md. Companion-material links open the repository originals.